Using HTTP cookies

An HTTP cookie (web cookie, browser cookie) is a **small piece of data that a server sends to a user's web browser**. The browser may store the cookie and send it back to the same server with later requests. Typically, an HTTP cookie is used to tell if two requests come from the same browser—keeping a user logged in, for example. It remembers stateful information for the stateless HTTP protocol.

Cookies are mainly used for three purposes:

1. Session management:
   - Logins, shopping carts, game scores, or anything else the server should remember
2. Personalization:
   - User preferences, themes, and other settings
3. Tracking:
   - Recording and analyzing user behavior

Cookies were once used for general client-side storage. While this made sense when they were the only way to store data on the client, modern storage APIs are now recommended. Cookies are sent with every request, so they can worsen performance (especially for mobile data connections). Modern APIs for client storage are the Web Storage API (`localStorage` and `sessionStorage`) and IndexedDB.

Note: To see stored cookies (and other storage that a web page can use), you can enable the Storage Inspector in Developer Tools and select Cookies from the storage tree.

Creating cookies

After receiving an HTTP request, a server can send one or more `Set-Cookie` headers with the response. The browser usually stores the cookie and sends it with requests made to the same server inside a `Cookie` HTTP header. You can specify an expiration date or time period after which the cookie shouldn't be sent. You can also set additional restrictions to a specific domain

and path to limit where the cookie is sent. For details about the header attributes mentioned below, refer to the `Set-Cookie` reference article.

The `Set-Cookie` and `Cookie` headers

The `Set-Cookie` HTTP response header sends cookies from the server to the user agent. A simple cookie is set like this:

```
Set-Cookie: <cookie-name>=<cookie-value>
```

This instructs the server sending headers to tell the client to store a pair of cookies:

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: yummy_cookie=choco
Set-Cookie: tasty_cookie=strawberry

[page content]
```

Then, with every subsequent request to the server, the browser sends all previously stored cookies back to the server using the `Cookie` header.

```
GET /sample_page.html HTTP/2.0
Host: www.example.org
Cookie: yummy_cookie=choco; tasty_cookie=strawberry
```

Define the lifetime of a cookie

The lifetime of a cookie can be defined in two ways:

- Session cookies are deleted when the current session ends. The browser defines when the "current session" ends, and some browsers use session restoring when restarting. This can cause session cookies to last indefinitely.
- Permanent cookies are deleted at a date specified by the `Expires` attribute, or after a period of time specified by the `Max-Age` attribute.

For example:

```
Set-Cookie: id=a3fWa; Expires=Thu, 31 Oct 2021 07:28:00 GMT;
```

Note: When you set an Expires date and time, they're relative to the client the cookie is being set on, not the server.

If your site authenticates users, it should regenerate and resend session cookies, even ones that already exist, whenever a user authenticates. This approach helps prevent session fixation attacks, where a third party can reuse a user's session.

Restrict access to cookies

You can ensure that cookies are sent securely and aren't accessed by unintended parties or scripts in one of two ways: with the `Secure` attribute and the `HttpOnly` attribute.

A cookie with the `Secure` attribute is only sent to the server with an encrypted request over the HTTPS protocol. It's never sent with unsecured HTTP (except on localhost), which means man-in-the-middle attackers can't access it easily. Insecure sites (with `http`: in the URL) can't set cookies with the Secure attribute. However, don't assume that `Secure` prevents all access to sensitive information in cookies. For example, someone with access to the client's hard disk (or JavaScript if the `HttpOnly` attribute isn't set) can read and modify the information.

A cookie with the `HttpOnly` attribute is inaccessible to the JavaScript `Document.cookie` API; it's only sent to the server. For example, cookies that persist in server-side sessions don't need to be available to JavaScript and should have the `HttpOnly` attribute. This precaution helps mitigate cross-site scripting (XSS) attacks.

Here's an example:

```
Set-Cookie: id=a3fWa; Expires=Thu, 21 Oct 2021 07:28:00 GMT; Secure; HttpOnly
```

Define where cookies are sent

The `Domain` and `Path` attributes define the scope of a cookie: what URLs the cookies should be sent to.

Domain attribute

The `Domain` attribute specifies which hosts can receive a cookie. If the server does not specify a `Domain`, the browser defaults the domain to the same host that set the cookie, excluding subdomains. If `Domain` is specified, then subdomains are always included. Therefore, specifying `Domain` is less restrictive than omitting it. However, it can be helpful when subdomains need to share information about a user.

For example, if you set `Domain=mozilla.org`, cookies are available on subdomains like `developer.mozilla.org`.

Path attribute

The `Path` attribute indicates a URL path that must exist in the requested URL in order to send the Cookie header. The `%x2F` ("/") character is considered a directory separator, and subdirectories match as well.

For example, if you set `Path=/docs`, these request paths match:

- `/docs`
- `/docs/`
- `/docs/Web/`
- `/docs/Web/HTTP`

But these request paths don't:

- `/`
- `/docsets`
- `/fr/docs`

SameSite attribute

The `SameSite` attribute lets servers specify whether/when cookies are sent with cross-site requests (where Site is defined by the registrable domain and the scheme: http or https). This provides some protection against cross-site request forgery attacks (CSRF). It takes three possible values: `Strict`, `Lax`, and `None`.

With `Strict`, the browser only sends the cookie with requests from the cookie's origin site. `Lax` is similar, except the browser also sends the cookie when the user navigates to the cookie's origin site (even if the user is coming from a different site). For example, by following a link from an external site. `None` specifies that cookies are sent on both originating and cross-site requests,

but only in secure contexts (i.e., if `SameSite=None` then the `Secure` attribute must also be set). If no `SameSite` attribute is set, the cookie is treated as `Lax`.

Here's an example:

```
Set-Cookie: mykey=myvalue; SameSite=Strict
```

Cookie prefixes

Because of the design of the cookie mechanism, a server can't confirm that a cookie was set from a secure origin or even tell where a cookie was originally set.

A vulnerable application on a subdomain can set a cookie with the Domain attribute, which gives access to that cookie on all other subdomains. This mechanism can be abused in a session fixation attack. See session fixation for primary mitigation methods.

As a defense-in-depth measure, however, you can use cookie prefixes to assert specific facts about the cookie. Two prefixes are available:

`__Host-`

If a cookie name has this prefix, it's accepted in a `Set-Cookie` header only if it's also marked with the `Secure` attribute, was sent from a secure origin, does not include a `Domain` attribute, and has the Path attribute set to /. This way, these cookies can be seen as "domain-locked".

`__Secure-`

If a cookie name has this prefix, it's accepted in a `Set-Cookie` header only if it's marked with the `Secure` attribute and was sent from a secure origin. This is weaker than the `__Host-` prefix.

The browser will reject cookies with these prefixes that don't comply with their restrictions. Note that this ensures that subdomain-created cookies with prefixes are either confined to the subdomain or ignored completely. As the application server only checks for a specific cookie name when determining if the user is authenticated or a CSRF token is correct, this effectively acts as a defense measure against session fixation.

JavaScript access using Document.cookie

You can create new cookies via JavaScript using the `Document.cookie` property. You can access existing cookies from JavaScript as well if the `HttpOnly` flag isn't set.

```
document.cookie = "yummy_cookie=choco";
document.cookie = "tasty_cookie=strawberry";
console.log(document.cookie);
// logs "yummy_cookie=choco; tasty_cookie=strawberry"
```

Cookies created via JavaScript can't include the `HttpOnly` flag.

Please note the security issues in the Security section below. Cookies available to JavaScript can be stolen through XSS.

Security

Note: When you store information in cookies, keep in mind that all cookie values are visible to, and can be changed by, the end user. Depending on the application, you may want to use an opaque identifier that the server looks up, or investigate alternative authentication/confidentiality mechanisms such as JSON Web Tokens.

Ways to mitigate attacks involving cookies:

- Use the `HttpOnly` attribute to prevent access to cookie values via JavaScript.
- Cookies that are used for sensitive information (such as indicating authentication) should have a short lifetime, with the `SameSite` attribute set to Strict or Lax. (See SameSite attribute, above.) In browsers that support SameSite, this ensures that the authentication cookie isn't sent with cross-site requests. This would make the request effectively unauthenticated to the application server.

Creating cookies

Third-party cookies

A cookie is associated with a particular domain and scheme (such as `http` or `https`), and may also be associated with subdomains if the `Set-Cookie Domain` attribute is set. If the cookie domain and scheme match the current page, the cookie is considered to be from the same site as the page, and is referred to as a first-party cookie.

If the domain and scheme are different, the cookie is not considered to be from the same site, and is referred to as a third-party cookie. While the server hosting a web page sets first-party cookies, the page may contain images or other components stored on servers in other domains (for example, ad banners) that may set third-party cookies. These are mainly used for advertising and tracking across the web. For example, the types of cookies used by Google.

A third-party server can create a profile of a user's browsing history and habits based on cookies sent to it by the same browser when accessing multiple sites. Firefox, by default, blocks third-party cookies that are known to contain trackers. Third-party cookies (or just tracking cookies) may also be blocked by other browser settings or extensions. Cookie blocking can cause some third-party components (such as social media widgets) not to function as intended.

There are some useful features available for developers who wish to respect user privacy, and minimize third-party tracking:

- Servers can (and should) set the cookie SameSite attribute to specify whether or not third-party cookies may be sent.
- Cookies Having Independent Partitioned State (CHIPS) enables developers to opt-in their cookies to partitioned storage, with a separate cookie jar per top-level site. This enables valid non-tracking uses of third-party cookies to continue working in browsers that do not allow cookies to be used for third-party tracking.

Cookie-related regulations

Legislation or regulations that cover the use of cookies include:

- The General Data Privacy Regulation (GDPR) in the European Union
- The ePrivacy Directive in the EU
- The California Consumer Privacy Act

These regulations have a global reach. They apply to any site on the World Wide Web that users from these jurisdictions access (the EU and California).

These regulations include requirements such as:

- Notifying users that your site uses cookies.
- Allowing users to opt out of receiving some or all cookies.
- Allowing users to use the bulk of your service without receiving cookies.

There may be other regulations that govern the use of cookies in your locality. The burden is on you to know and comply with these regulations. There are companies that offer "cookie banner" code that helps you comply with these regulations.

Other ways to store information in the browser

Another approach to storing data in the browser is the Web Storage API. The window.sessionStorage and window.localStorage properties correspond to session and permanent cookies in duration, but have larger storage limits than cookies, and are never sent to a server. More structured and larger amounts of data can be stored using the IndexedDB API, or a library built on it.

There are some techniques designed to recreate cookies after they're deleted. These are known as "zombie" cookies. These techniques violate the principles of user privacy and user control, may violate data privacy regulations, and could expose a website using them to legal liability.

This lesson was adapted from MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies